Guía didáctica

Directorio de Contactos

Explicación línea por línea de cada archivo

Python (Flask) · MySQL · HTML · CSS

¿Para quién es esta guía?

Para quien está dando sus primeros pasos en programación. No se necesita experiencia previa: cada línea de código está explicada con palabras sencillas y ejemplos cotidianos. Las filas en **verde** y marcadas como NUEVO son las que se agregaron para poder **seleccionar y modificar** un contacto.

1. Antes de empezar: ¿qué hace este programa?

Es una pequeña **página web** que funciona como agenda. Permite tres cosas: **ver** los contactos guardados, **agregar** uno nuevo y (con el código nuevo) **seleccionar uno y modificarlo**. Los datos no se pierden al cerrar el navegador porque se guardan en una base de datos.

Una analogía: el restaurante

Imagina que usas la página como si estuvieras en un restaurante:

- **Tú y tu navegador** (Chrome, Edge...) = el cliente que mira el menú y hace pedidos.
- **Flask (`app.py`)** = el mesero. Recibe tu pedido, lo lleva a la cocina y te trae la respuesta.
- **MySQL** = la bodega/cocina donde se guardan los ingredientes (los contactos).
- **HTML (`index.html`)** = el menú impreso: define *qué* aparece en pantalla.
- **CSS (`style.css`)** = la decoración del restaurante: define *cómo se ve* (colores, tamaños, bordes).



La respuesta hace el camino de regreso: MySQL → Flask → navegador.

Estructura de carpetas

Flask exige que los archivos estén en carpetas con nombres específicos:

```
directorio/
├── app.py              <- el programa principal (Python)
├── templates/
│   └── index.html      <- la página (HTML)
└── static/
    └── style.css       <- los estilos (CSS)
```

Cómo ejecutarlo

- Instala las librerías (una sola vez), escribiendo en la terminal: `pip install flask mysql-connector-python`
- Asegúrate de que MySQL esté encendido y de que el usuario y contraseña de `DB_CONFIG` sean correctos.
- Dentro de la carpeta del proyecto ejecuta: `python app.py`
- Abre el navegador en `http://127.0.0.1:5000`

¿Cómo funciona "modificar un contacto"? (paso a paso)

- Abres la página principal. `app.py` (ruta `/`) lee todos los contactos de MySQL y se los pasa a `index.html`, que dibuja la tabla con un botón **Editar** en cada fila.
- Haces clic en **Editar** del contacto #3. El navegador visita la dirección `/editar/3`.

- La ruta `editar` de `app.py` busca en MySQL el contacto con id 3 y muestra otra vez `index.html`, pero ahora con el contacto seleccionado.
- `index.html` detecta que hay un contacto seleccionado: cambia el título, **prellena** el formulario con sus datos y resalta su fila en amarillo.
- Corriges lo que quieras y pulsas **Actualizar**. El formulario envía los datos a `/actualizar/3`.
- La ruta `actualizar` de `app.py` ejecuta un `UPDATE` en MySQL, guarda y te regresa a la página principal con el cambio ya visible.

Cómo leer las tablas de las siguientes páginas
Línea: número de línea en el archivo. **Código:** lo que está escrito. **¿Qué hace?:** explicación en lenguaje sencillo. Las filas verdes marcadas **NUEVO** son el código agregado para editar contactos. Las líneas en blanco no se explican porque solo sirven para separar y que el código sea legible.

2. Archivo app.py (el cerebro, en Python)

Este archivo es el programa principal. Se conecta a MySQL, crea la base de datos y define las "rutas": direcciones web que hacen algo cuando alguien las visita. Toda la lógica pasa aquí. **Sobre la sangría:** en Python, los espacios al inicio de una línea son obligatorios; indican qué líneas pertenecen a una función, un `if` o un `try`.

Línea	Código	¿Qué hace? (explicación)
1	`# app.py`	Un **comentario**. En Python, todo lo que va después de `#` es ignorado por la computadora: son notas para las personas que leen el código. Aquí solo dice el nombre del archivo.

2–6	`# ------------------------------------------------------------`
	`# Programa en Flask que se conecta a MySQL, crea la base`
	`# de datos y tabla si no existen, y permite dar de alta`
	`# registros en el directorio.`
	`# ------------------------------------------------------------`
		Más comentarios. Las líneas con guiones (`------`) son solo decoración para separar visualmente. El texto explica que el programa usa **Flask** (para la página web) y **MySQL** (para guardar datos). Como ahora el programa también *modifica* contactos, podrías agregar esa palabra al comentario.

8 NUEVO	`from flask import Flask, render_template, request, redirect, url_for, abort`	`import` significa "traer herramientas de otra caja". Con `from flask import ...` traemos 6 herramientas de la librería Flask: • `Flask`: crea la aplicación web. • `render_template`: muestra un archivo HTML. • `request`: contiene lo que el usuario envió (por ejemplo, lo que escribió en el formulario). • `redirect`: manda al navegador a otra dirección. • `url_for`: construye una dirección a partir del nombre de una función. • `abort`: detiene todo y muestra un error (por ejemplo, 404 = "no encontrado"). `abort` **es la herramienta nueva** que agregamos.
9	`import mysql.connector`	Trae la librería `mysql.connector`, que es el "traductor" que permite a Python hablar con MySQL.
10	`from mysql.connector import Error`	Trae específicamente la clase `Error` de esa librería. La usaremos más abajo para atrapar problemas con la base de datos.
12	`app = Flask(__name__)`	Crea la aplicación y la guarda en una **variable** llamada `app`. Una variable es una cajita con nombre donde se guarda un dato. `__name__` es el nombre del archivo actual; Flask lo usa para saber dónde buscar las carpetas `templates` y `static`.

14–16	`# ------------------------------------------------------------`
	`# DATOS DE CONEXIÓN A MYSQL`
	`# ------------------------------------------------------------`
		Comentario que marca la sección de configuración de la base de datos.

Línea	Código	¿Qué hace? (explicación)
17	`DB_CONFIG = {`	Crea un **diccionario** llamado `DB_CONFIG`. Un diccionario es como una agenda: cada dato tiene una etiqueta (clave) y un valor, separados por dos puntos. Las llaves `{ }` lo encierran. El nombre va en MAYÚSCULAS por convención: indica que es una configuración que no cambiará.
18	`    "host": "localhost",`	Clave `host` (dónde está MySQL). `localhost` significa "esta misma computadora".
19	`    "user": "coram",`	Clave `user`: el usuario con el que entramos a MySQL.
20	`    "password": "coramadministrador2008",`	Clave `password`: la contraseña de ese usuario. **Cuidado:** escribir contraseñas dentro del código es riesgoso si compartes el archivo (ver recomendaciones al final).
21	`    "database": "directorio"`	Clave `database`: el nombre de la base de datos que usaremos. No lleva coma al final porque es el último elemento.
22	`}`	Cierra el diccionario con `}`.
25	`def inicializar_bd():`	`def` significa **definir una función**: una "receta" con nombre que se ejecuta solo cuando alguien la llama. Esta receta se llama `inicializar_bd` y no necesita datos de entrada (paréntesis vacíos). Los dos puntos `:` indican que lo que sigue (con sangría) es el contenido de la receta.
26	`    """Crea la base de datos y la tabla si no existen."""`	Un **docstring**: texto entre triples comillas que describe qué hace la función. Sirve de documentación.
27	`    try:`	`try` significa "intenta". Python prueba ejecutar el bloque de abajo; si algo falla, en lugar de que el programa se caiga salta al bloque `except` (línea 51).
28	`        conexion = mysql.connector.connect(`	Crea una **conexión** con MySQL y la guarda en la variable `conexion`. Piensa en esto como marcar un número telefónico: aún no hablamos, pero ya hay línea.
29	`            host=DB_CONFIG["host"],`	Le decimos en qué computadora está MySQL. `DB_CONFIG["host"]` busca en el diccionario el valor de la etiqueta `host`.
30	`            user=DB_CONFIG["user"],`	Le decimos con qué usuario entrar.
31	`            password=DB_CONFIG["password"]`	Le damos la contraseña. Observa que **no** ponemos la base de datos aquí: todavía no sabemos si existe, ¡justo la vamos a crear!
32	`        )`	Cierra el paréntesis de `connect(`.
33	`        cursor = conexion.cursor()`	Crea un **cursor**. Es el "mensajero" que lleva nuestras órdenes en lenguaje SQL hasta MySQL y trae las respuestas.
34	`        # Crear base de datos si no existe`	Comentario que explica la siguiente línea.

Línea	Código	¿Qué hace? (explicación)
35	`cursor.execute("CREATE DATABASE IF NOT EXISTS directorio")`	Primera orden SQL: `CREATE DATABASE IF NOT EXISTS directorio` = "crea la base de datos llamada directorio, pero solo si todavía no existe". El `IF NOT EXISTS` evita errores al ejecutar el programa varias veces.
36	`cursor.execute("USE directorio")`	`USE directorio` = "entra a la base de datos directorio" para trabajar dentro de ella.
37	`# Crear tabla si no existe`	Comentario.
38	`cursor.execute("""`	Ejecuta una orden SQL larga. Las **triples comillas** `"""` permiten escribir texto en varias líneas.
39	`CREATE TABLE IF NOT EXISTS contactos (`	`CREATE TABLE IF NOT EXISTS contactos (` = "crea una tabla llamada contactos si no existe". Una tabla es como una hoja de Excel: columnas (campos) y filas (registros). Dentro del paréntesis van las columnas.
40	`id INT AUTO_INCREMENT PRIMARY KEY,`	Columna `id`: número entero (`INT`). `AUTO_INCREMENT` hace que MySQL le ponga 1, 2, 3... automáticamente. `PRIMARY KEY` significa que es el identificador único de cada contacto, como una CURP: no se repite.
41	`nombre VARCHAR(100) NOT NULL,`	Columna `nombre`: texto de hasta 100 caracteres (`VARCHAR(100)`). `NOT NULL` significa que es obligatorio.
42	`telefono VARCHAR(20),`	Columna `telefono`: texto de hasta 20 caracteres. Se guarda como texto (no número) para permitir guiones, `+` o ceros iniciales.
43	`correo VARCHAR(100),`	Columna `correo`: texto de hasta 100 caracteres.
44	`fecha_registro TIMESTAMP DEFAULT CURRENT_TIMESTAMP`	Columna `fecha_registro`: guarda fecha y hora (`TIMESTAMP`). `DEFAULT CURRENT_TIMESTAMP` la llena sola con el momento en que se crea el contacto.
45	`)`	Cierra el paréntesis de las columnas.
46	`""")`	Cierra las triples comillas y el paréntesis de `execute(`.
47	`conexion.commit()`	`commit()` = **confirmar y guardar** los cambios de forma definitiva. Sin esto, MySQL podría descartarlos.
48	`cursor.close()`	Cierra el cursor (el mensajero ya terminó su trabajo).
49	`conexion.close()`	Cierra la conexión (colgamos el teléfono). Es buena práctica para no dejar conexiones abiertas.
50	`print("Base de datos y tabla listas.")`	`print` muestra un mensaje en la terminal. La `f` antes de las comillas se usa en la línea 52; aquí es texto simple.
51	`except Error as e:`	Si algo dentro del `try` falló con un `Error` de MySQL, se ejecuta esto. `as e` guarda el detalle del error en la variable `e`.

Línea	Código	¿Qué hace? (explicación)
52	`print(f"Error al inicializar la BD: {e}")`	Muestra en la terminal el mensaje de error. La `f` antes de las comillas crea un **f-string**: lo que va entre llaves `{e}` se reemplaza por el valor de la variable.
55	`def obtener_conexion():`	Otra función: `obtener_conexion`. Será la que llamemos cada vez que necesitemos hablar con la base de datos.
56	`"""Crea y devuelve una conexión a la base de datos MySQL."""`	Docstring que describe la función.
57	`conexion = mysql.connector.connect(**DB_CONFIG)`	Crea la conexión. Los dos asteriscos `**` "desempacan" el diccionario: es lo mismo que escribir `host=..., user=..., password=..., database=...` pero más corto. Aquí sí incluimos `database`, porque ya existe.
58	`return conexion`	`return` **devuelve** el resultado a quien llamó la función. Es como una tienda que te entrega tu compra.
61	`@app.route("/")`	Un **decorador**. La línea `@app.route("/")` dice: "cuando alguien visite la dirección principal (`/`), ejecuta la función que está justo debajo".
62	`def index():`	Define la función `index` (la página principal).
63	`"""Página principal: formulario y lista de registros."""`	Docstring de la función.
64	`conexion = obtener_conexion()`	Abre una conexión llamando a la función de la línea 55.
65	`cursor = conexion.cursor(dictionary=True)`	Crea el cursor. `dictionary=True` hace que cada fila se devuelva como diccionario (`{'id': 1, 'nombre': 'Ana', ...}`), y así en el HTML podemos escribir `c.nombre`.
66	`cursor.execute("SELECT * FROM contactos ORDER BY id DESC")`	Orden SQL: `SELECT *` = "tráeme todas las columnas", `FROM contactos` = "de la tabla contactos", `ORDER BY id DESC` = "ordenados por id de mayor a menor" (el más reciente primero).
67	`contactos = cursor.fetchall()`	`fetchall()` recoge **todas** las filas de la respuesta y las guarda en la lista `contactos`.
68-69	`cursor.close()` `conexion.close()`	Cierra el cursor y la conexión.
70 NUEVO	`return render_template("index.html", contactos=contactos, contacto_editar=None)`	Envía la información al archivo HTML. `render_template` toma `index.html`, le entrega `contactos` (la lista) y **ahora también** `contacto_editar=None`, que significa "no hay ningún contacto seleccionado para modificar". `None` es el valor de "nada" en Python.
73 NUEVO	`@app.route("/editar/<int:id>")`	**Ruta nueva.** Se activa con direcciones como `/editar/3`. La parte `<int:id>` es una variable en la dirección: `int` obliga a que sea un número entero (si escriben `/editar/hola` Flask responde 404) y se guarda con el nombre `id`.
74 NUEVO	`def editar(id):`	La función recibe ese `id` como parámetro: si la dirección fue `/editar/3`, aquí `id` vale 3.

Línea	Código	¿Qué hace? (explicación)
75 NUEVO	`"""Selecciona un contacto y lo carga en el formulario para modificarlo."""`	Docstring: esta función selecciona un contacto y lo manda al formulario.
76–77 NUEVO	`conexion = obtener_conexion()` `cursor = conexion.cursor(dictionary=True)`	Abre la conexión y crea el cursor, igual que en `index`.
79 NUEVO	`# Contacto seleccionado`	Comentario.
80 NUEVO	`cursor.execute("SELECT * FROM contactos WHERE id = %s", (id,))`	Busca **un solo contacto**: `WHERE id = %s` significa "donde el id sea...". El `%s` es un **marcador de posición**; el valor real se envía aparte en `(id,)`. Esto protege contra ataques de *inyección SQL*. Nota la coma en `(id,)`: en Python, una tupla de un solo elemento la necesita.
81 NUEVO	`contacto_editar = cursor.fetchone()`	`fetchone()` trae **una sola fila** (o `None` si no encontró nada) y la guarda en `contacto_editar`.
83 NUEVO	`# Lista completa (para seguir mostrando la tabla)`	Comentario.
84–85 NUEVO	`cursor.execute("SELECT * FROM contactos ORDER BY id DESC")` `contactos = cursor.fetchall()`	Trae también la lista completa, porque la página siempre muestra el formulario **y** la tabla.
87–88 NUEVO	`cursor.close()` `conexion.close()`	Cierra el cursor y la conexión.
90 NUEVO	`if contacto_editar is None:`	`if` = "si". Si `contacto_editar` es `None`, el id no existe en la base de datos.
91 NUEVO	`abort(404) # el id no existe`	`abort(404)` detiene todo y muestra el error "404 No encontrado". La sangría (espacios al inicio) indica que esta línea pertenece al `if`. El comentario tras `#` es una nota.
93–94 NUEVO	`return render_template("index.html", contactos=contactos,` `contacto_editar=contacto_editar)`	Muestra la página enviando la lista y el contacto seleccionado. La segunda línea es continuación de la primera (Python permite partir una línea larga dentro de paréntesis).
97 NUEVO	`@app.route("/actualizar/<int:id>", methods=["POST"])`	**Ruta nueva** que recibe los cambios. `methods=["POST"]` significa que solo acepta datos **enviados por un formulario** (no se puede abrir escribiendo la dirección en el navegador).
98 NUEVO	`def actualizar(id):`	Define la función y recibe el `id` del contacto que se va a modificar.
99 NUEVO	`"""Guarda los cambios del contacto seleccionado (UPDATE)."""`	Docstring. `UPDATE` es la palabra SQL para "actualizar".
100 NUEVO	`nombre = request.form["nombre"]`	Lee lo que el usuario escribió en el campo con `name="nombre"` del formulario y lo guarda en la variable `nombre`.

Línea	Código	¿Qué hace? (explicación)
101–102 NUEVO	`    telefono = request.form["telefono"]` `    correo = request.form["correo"]`	Igual para `telefono` y `correo`.
104–105 NUEVO	`    conexion = obtener_conexion()` `    cursor = conexion.cursor()`	Abre la conexión y crea el cursor.
106 NUEVO	`    sql = "UPDATE contactos SET nombre = %s, telefono = %s, correo = %s WHERE id = %s"`	Orden SQL: "actualiza la tabla contactos, cambia nombre, telefono y correo por los nuevos valores, **solo en la fila cuyo id sea...**". ¡El `WHERE` es importantísimo! Sin él, cambiarías **todos** los contactos.
107 NUEVO	`    cursor.execute(sql, (nombre, telefono, correo, id))`	Ejecuta la orden. Los cuatro `%s` se rellenan en orden con lo que hay en la tupla `(nombre, telefono, correo, id)`.
108 NUEVO	`    conexion.commit()`	`commit()` guarda los cambios definitivamente.
109–110 NUEVO	`    cursor.close()` `    conexion.close()`	Cierra cursor y conexión.
112 NUEVO	`    return redirect(url_for("index"))`	`redirect(url_for("index"))` manda al navegador de regreso a la página principal, que ya mostrará los datos actualizados. Esto evita que, si refrescas la página, se reenvíe el formulario.
115	`@app.route("/agregar", methods=["POST"])`	Ruta `/agregar`, solo por POST. Es la que recibe el formulario cuando estamos **creando** un contacto nuevo.
116–117	`def agregar():` `    """Inserta un nuevo registro en la tabla contactos."""`	Define la función `agregar` y su docstring.

118–120	`    nombre = request.form["nombre"]`
	`    telefono = request.form["telefono"]`
	`    correo = request.form["correo"]`
		Lee los tres campos que llegaron del formulario: nombre, teléfono y correo.

122–123	`    conexion = obtener_conexion()` `    cursor = conexion.cursor()`	Abre conexión y cursor.
124	`    sql = "INSERT INTO contactos (nombre, telefono, correo) VALUES (%s, %s, %s)"`	Orden SQL `INSERT INTO`: "inserta en la tabla contactos, en las columnas nombre, telefono y correo, estos tres valores". Los `%s` son marcadores de posición seguros.
125	`    valores = (nombre, telefono, correo)`	Junta los tres datos en una **tupla** (una lista que no se puede modificar), en el mismo orden que los `%s`.
126	`    cursor.execute(sql, valores)`	Ejecuta la orden con sus valores.
127	`    conexion.commit()`	Guarda los cambios (`commit`).
128–129	`    cursor.close()` `    conexion.close()`	Cierra cursor y conexión.
131	`    return redirect(url_for("index"))`	Regresa a la página principal para ver el nuevo contacto.

Línea	Código	¿Qué hace? (explicación)
134	`if __name__ == "__main__":`	Significa: "solo si este archivo se ejecuta directamente (`python app.py`), haz lo siguiente". Si otro programa importara este archivo, esto no se ejecutaría.
135	`    inicializar_bd() # Se asegura de que la BD y tabla existan`	Llama a la función que crea la base de datos y la tabla. Por eso funciona aunque MySQL esté vacío.
136	`    app.run(debug=True)`	Arranca el servidor web. `debug=True` activa el **modo depuración**: el servidor se reinicia solo al guardar cambios y muestra errores detallados. Úsalo solo mientras programas, nunca en un sitio público.

3. Archivo templates/index.html (la página que se ve)

HTML es el lenguaje que describe el contenido de una página con **etiquetas** como `<h1>` o `<table>`. La mayoría se abre y se cierra (`<p>` ... `</p>`). Este archivo es además una **plantilla**: mezcla HTML con instrucciones de Jinja. Regla fácil: `{{ ... }}` **muestra** un valor; `{% ... %}` ejecuta una **instrucción** (`if`, `for`) y no se ve. La sangría (espacios al inicio) en HTML es solo para ordenar; aquí se acortó para que quepa en la tabla.

Línea	Código	¿Qué hace? (explicación)
1	`<!DOCTYPE html>`	Le dice al navegador: "este documento es HTML5". Siempre va en la primera línea.
2	`<html lang="es">`	Abre la etiqueta `<html>`, que envuelve toda la página. `lang="es"` indica que el contenido está en español (ayuda a lectores de pantalla y traductores).
3	`<head>`	Abre `<head>` (la "cabeza"): información *sobre* la página que no se ve directamente.
4	`<meta charset="UTF-8">`	`charset="UTF-8"` permite mostrar acentos, ñ y emojis correctamente.
5	`<title>Directorio de Contactos</title>`	`<title>`: el texto que aparece en la pestaña del navegador.
6	`<link rel="stylesheet" href="{{ url_for('static', filename='style.css') }}">`	Conecta el archivo de estilos CSS. Lo que está entre `{{ }}` no es HTML: es **Jinja**, el lenguaje de plantillas de Flask. `url_for('static', filename='style.css')` calcula la dirección correcta del archivo dentro de la carpeta `static`.
7	`</head>`	Cierra la cabeza.
8	`<body>`	Abre `<body>` (el "cuerpo"): todo lo que se ve en pantalla.
10	`<div class="contenedor">`	Abre un `<div>`, una caja genérica para agrupar cosas. `class="contenedor"` le pone una etiqueta para que el CSS pueda darle estilo (línea 12 del CSS).
11	`<h1>(emoji) Directorio de Contactos</h1>`	`<h1>` es el título principal. El emoji de agenda es solo un carácter más, como una letra (en este PDF se muestra como "(emoji)" porque el tipo de letra usado no lo incluye).
13	`<!-- ==================== FORMULARIO ==================== -->`	Comentario HTML: se escribe entre `<!--` y `-->` y no se muestra en pantalla.
14 NUEVO	`{% if contacto_editar %}`	**Lógica Jinja.** Lo que va entre `{% %}` son instrucciones, no se muestra. `{% if contacto_editar %}` = "si hay un contacto seleccionado para modificar...". Recuerda que Flask nos manda `None` cuando no hay ninguno.
15 NUEVO	`<h2>Modificar contacto #{{ contacto_editar.id }}</h2>`	Título de la sección en modo edición. `{{ contacto_editar.id }}` **imprime** el valor: por ejemplo, "Modificar contacto #3".

Línea	Código	¿Qué hace? (explicación)
16 NUEVO	`<form action="{{ url_for('actualizar', id=contacto_editar.id) }}" method="POST" class="formulario">`	Abre el formulario en modo edición. `action` es la dirección a donde se enviarán los datos: `url_for('actualizar', id=...)` genera algo como `/actualizar/3`. `method="POST"` envía los datos de forma oculta (no en la dirección). `class="formulario"` es para el CSS.
17 NUEVO	`{% else %}`	`{% else %}` = "si no hay contacto seleccionado..." (es decir, estamos agregando uno nuevo).
18	`<h2>Agregar nuevo contacto</h2>`	Título en modo "agregar".
19	`<form action="{{ url_for('agregar') }}" method="POST" class="formulario">`	Abre el formulario en modo agregar, apuntando a `/agregar`.
20 NUEVO	`{% endif %}`	`{% endif %}` cierra el `if`. El truco: solo **uno** de los dos `<form>` (línea 16 o 19) llega al navegador. Ambos se cierran con el único `</form>` de la línea 39.
22	`<label for="nombre">Nombre:</label>`	`<label>` es la etiqueta de texto de un campo. `for="nombre"` la enlaza con el campo que tenga `id="nombre"`: al hacer clic en la etiqueta, el cursor entra en el campo.
23	`<input type="text" id="nombre" name="nombre" required`	`<input>` es una caja de escritura. `type="text"` = texto normal. `id` sirve para enlazar con la etiqueta. `name="nombre"` **es clave:** es el nombre con el que Python (`request.form["nombre"]`) recibe el dato. `required` obliga a llenar el campo antes de enviar.
24 NUEVO	`value="{{ contacto_editar.nombre if contacto_editar else '' }}">`	`value="..."` es el texto que aparece escrito de inicio. `{{ contacto_editar.nombre if contacto_editar else '' }}` significa: "si hay contacto seleccionado, pon su nombre; si no, déjalo vacío". Así el formulario aparece **prellenado** al editar.
26	`<label for="telefono">Teléfono:</label>`	Etiqueta del campo Teléfono.
27	`<input type="text" id="telefono" name="telefono" required`	Campo de texto para el teléfono (`name="telefono"`), obligatorio.
28 NUEVO	`value="{{ contacto_editar.telefono if contacto_editar else '' }}">`	Prellena el teléfono si estamos editando (misma idea que la línea 24).
30	`<label for="correo">Correo electrónico:</label>`	Etiqueta del campo Correo.
31	`<input type="email" id="correo" name="correo" required`	Campo para el correo. `type="email"` hace que el navegador verifique que tenga forma de correo (con `@`).
32 NUEVO	`value="{{ contacto_editar.correo if contacto_editar else '' }}">`	Prellena el correo si estamos editando.
34 NUEVO	`<button type="submit">{{ 'Actualizar' if contacto_editar else 'Guardar' }}</button>`	El botón que envía el formulario (`type="submit"`). El texto cambia según el modo: dice "Actualizar" al editar y "Guardar" al agregar (mismo truco `if ... else ...` en una sola línea).

Línea	Código	¿Qué hace? (explicación)
36 NUEVO	`{% if contacto_editar %}`	Si estamos editando, se dibuja lo siguiente...
37 NUEVO	`<a href="{{ url_for('index') }}" class="cancelar">Cancelar</a>`	Un enlace (`<a>`) llamado "Cancelar" que regresa a la página principal (`url_for('index')`), descartando la edición.
38 NUEVO	`{% endif %}`	Cierra el `if`.
39	`</form>`	Cierra el formulario.
41	`<!-- ==================== LISTA DE CONTACTOS ==================== -->`	Comentario: inicio de la sección de la lista.
42	`<h2>Contactos guardados</h2>`	Subtítulo de la sección.
44	`{% if contactos %}`	`{% if contactos %}` = "si la lista de contactos no está vacía...".
45	`<table class="tabla">`	Abre una tabla (`<table>`) con la clase `tabla` para darle estilo.
46	`<tr>`	Abre una fila (`<tr>` = table row) de encabezados.
47–50	`<th>ID</th>` `<th>Nombre</th>` `<th>Teléfono</th>` `<th>Correo</th>`	Cuatro celdas de encabezado (`<th>` = table header): ID, Nombre, Teléfono y Correo.
51 NUEVO	`<th>Acciones</th>`	Encabezado nuevo: **Acciones**, donde irá el botón de editar.
52	`</tr>`	Cierra la fila de encabezados.
53	`{% for c in contactos %}`	**Ciclo** `for`: repite lo que sigue una vez por cada contacto de la lista. En cada vuelta, `c` representa el contacto actual (como leer una hoja de Excel fila por fila).
54 NUEVO	`<tr {% if contacto_editar and contacto_editar.id == c.id %}class="seleccionado"{% endif %}>`	Abre una fila de datos. Si el contacto de esta vuelta es el que estamos editando (mismo id), se le agrega `class="seleccionado"`, que el CSS pinta de amarillo. Es una condición dentro de una etiqueta.
55–58	`<td>{{ c.id }}</td>` `<td>{{ c.nombre }}</td>` `<td>{{ c.telefono }}</td>` `<td>{{ c.correo }}</td>`	Cuatro celdas de datos (`<td>` = table data) que **imprimen** el id, nombre, teléfono y correo del contacto actual.
59 NUEVO	`<td><a href="{{ url_for('editar', id=c.id) }}" class="btn-editar">(emoji) Editar</a></td>`	Celda nueva con un enlace hacia `/editar/<id>` (por ejemplo `/editar/3`) usando `url_for('editar', id=c.id)`. Al hacer clic, se ejecuta la ruta `editar` de `app.py`. La clase `btn-editar` lo hace lucir como botón naranja.
60	`</tr>`	Cierra la fila.
61	`{% endfor %}`	Fin del ciclo `for`: aquí regresa al inicio (línea 53) hasta terminar todos los contactos.
62	`</table>`	Cierra la tabla.
63	`{% else %}`	`{% else %}` = "si la lista está vacía..."

Línea	Código	¿Qué hace? (explicación)
64	`<p>No hay contactos registrados todavía.</p>`	Muestra un párrafo (`<p>`) con un aviso amable.
65	`{% endif %}`	Cierra el `if` de la línea 44.
67	`</div>`	Cierra la caja `contenedor` abierta en la línea 10.
69	`</body>`	Cierra el cuerpo de la página.
70	`</html>`	Cierra el documento HTML.

4. Archivo static/style.css (el diseño)

CSS está formado por **reglas**. Cada regla tiene un **selector** (a quién le aplico el estilo), llaves `{ }` y dentro **propiedades** con esta forma: `propiedad: valor;`. Un selector con punto (`.tabla`) se refiere a los elementos con esa `class` en el HTML; uno sin punto (`body`, `h1`) se refiere a todas las etiquetas de ese tipo.

Línea	Código	¿Qué hace? (explicación)
1	`/* style.css - Estilos básicos para el directorio */`	Comentario CSS: se escribe entre `/*` y `*/`.
3	`body {`	**Selector** `body`: las reglas que siguen se aplican a todo el cuerpo de la página. Cada regla se abre con `{`.
4	`font-family: Arial, sans-serif;`	Tipo de letra: Arial; si no está disponible, cualquier fuente sin adornos (`sans-serif`).
5	`background-color:#d7d7f4;`	Color de fondo de toda la página (lila claro). Los colores se escriben en **hexadecimal**: `#` + 6 dígitos (rojo, verde, azul).
6–7	`margin: 0;` `padding: 0;`	Quita los márgenes y espacios internos por defecto del navegador.
8	`display: flex;`	`display: flex` activa el modo "flexible": una forma moderna de acomodar cosas.
9	`justify-content: center;`	Con `flex`, esto **centra horizontalmente** el contenido: la tarjeta queda al centro de la pantalla.
10	`}`	Cierra la regla con `}`.
12	`.contenedor {`	Selector de **clase**: el punto `.` significa "todo lo que tenga class='contenedor'" (la caja del HTML).
13	`background-color: #cf5e5e;`	Color de fondo de la tarjeta (un rojo suave).
14	`max-width: 600px;`	Ancho **máximo** de 600 píxeles; en pantallas grandes no se estira más.
15	`width: 100%;`	Pero usa el 100% del ancho disponible cuando la pantalla es pequeña (celulares).
16	`margin: 40px 20px;`	Margen exterior: 40 px arriba y abajo, 20 px a izquierda y derecha.
17	`padding: 30px;`	Espacio interior de 30 px entre el borde y el contenido.
18	`border-radius: 10px;`	Esquinas redondeadas de 10 px.
19	`box-shadow: 0 2px 8px rgba(0, 0, 0, 0.1);`	Sombra: `0` desplazamiento horizontal, `2px` vertical, `8px` de difuminado y un negro con 10% de opacidad (`rgba(0,0,0,0.1)`).
20	`}`	Cierra la regla.
22	`h1 {`	Selector `h1`: aplica a todos los títulos principales.
23	`text-align: center;`	Texto centrado.

Línea	Código	¿Qué hace? (explicación)
24	`color: #487aad;`	Color azul del texto.
25	`}`	Cierra la regla.
27	`h2 {`	Selector `h2`: aplica a los subtítulos.
28	`color: #34495e;`	Color gris azulado oscuro.
29	`border-bottom: 2px solid #eee;`	Línea inferior de 2 px, sólida, gris muy claro.
30	`padding-bottom: 5px;`	5 px de espacio entre el texto y esa línea.
31	`margin-top: 30px;`	30 px de espacio arriba del subtítulo.
32	`}`	Cierra la regla.
34	`.formulario {`	Clase `.formulario` (el `<form>`).
35	`display: flex;`	Activa el modo flexible.
36	`flex-direction: column;`	Acomoda los elementos en **columna** (uno debajo del otro) en vez de en fila.
37	`}`	Cierra la regla.
39	`.formulario label {`	Selector **descendiente**: `.formulario label` = "los `label` que estén *dentro* de `.formulario`".
40	`margin-top: 10px;`	10 px de espacio arriba de cada etiqueta.
41	`font-weight: bold;`	Texto en negritas.
42	`color: #555;`	Color gris.
43	`}`	Cierra la regla.
45	`.formulario input {`	Aplica a los campos `input` dentro del formulario.
46	`padding: 8px;`	Espacio interior de 8 px para que el texto no toque el borde.
47	`margin-top: 5px;`	5 px de separación arriba.
48	`border: 1px solid #ccc;`	Borde de 1 px, sólido, gris.
49	`border-radius: 5px;`	Esquinas redondeadas.
50	`}`	Cierra la regla.
52	`.formulario button {`	Aplica al botón del formulario.
53	`margin-top: 20px;`	20 px de espacio arriba.
54	`padding: 10px;`	Espacio interior de 10 px.
55	`background-color: #3498db;`	Fondo azul.
56	`color: white;`	Texto blanco.
57	`border: none;`	Sin borde.

Línea	Código	¿Qué hace? (explicación)
58	`border-radius: 5px;`	Esquinas redondeadas.
59	`cursor: pointer;`	Al pasar el mouse, el cursor cambia a una manita para indicar que se puede hacer clic.
60	`font-size: 16px;`	Tamaño de letra de 16 px.
61	`}`	Cierra la regla.
63	`.formulario button:hover {`	`:hover` es una **pseudoclase**: aplica solo mientras el mouse está encima del botón.
64	`background-color: #2980b9;`	El azul se oscurece un poco al pasar el mouse.
65	`}`	Cierra la regla.
67	`.tabla {`	Clase `.tabla` (la tabla de contactos).
68	`width: 100%;`	Ocupa todo el ancho disponible.
69	`border-collapse: collapse;`	Une los bordes de las celdas en una sola línea (sin doble borde).
70	`margin-top: 10px;`	10 px de espacio arriba.
71	`}`	Cierra la regla.
73	`.tabla th, .tabla td {`	Dos selectores separados por coma: la regla aplica a **ambos** (`th` y `td` dentro de `.tabla`).
74	`border: 1px solid #ddd;`	Borde gris claro en cada celda.
75	`padding: 8px;`	Espacio interior de 8 px.
76	`text-align: left;`	Texto alineado a la izquierda.
77	`}`	Cierra la regla.
79	`.tabla th {`	Solo los encabezados `th` de la tabla.
80	`background-color: #3498db;`	Fondo azul.
81	`color: white;`	Texto blanco.
82	`}`	Cierra la regla.
84	`.tabla tr:nth-child(even) {`	`tr:nth-child(even)` = "las filas pares (2, 4, 6...)". Es el truco de las filas "cebra".
85	`background-color: #f2f2f2;`	Les pone fondo gris muy claro para leer mejor.
86	`}`	Cierra la regla.
88 NUEVO	`/* ---------- Edición de contactos ---------- */`	Comentario: a partir de aquí están los estilos agregados para editar.
90 NUEVO	`.btn-editar {`	Estilo del botón **Editar** (el enlace de cada fila).

Línea	Código	¿Qué hace? (explicación)
91 NUEVO	`display: inline-block;`	`inline-block` deja el enlace en la misma línea, pero permite darle espacio interior como a una caja.
92 NUEVO	`padding: 4px 10px;`	4 px arriba/abajo y 10 px a los lados.
93 NUEVO	`background-color: #f39c12;`	Fondo naranja.
94 NUEVO	`color: white;`	Texto blanco.
95 NUEVO	`text-decoration: none;`	Quita el subrayado típico de los enlaces.
96 NUEVO	`border-radius: 5px;`	Esquinas redondeadas.
97 NUEVO	`font-size: 14px;`	Letra de 14 px.
98 NUEVO	`}`	Cierra la regla.

100–102 NUEVO	`.btn-editar:hover {`
	`  background-color: #d68910;`
	`}`
		Al pasar el mouse por encima, el naranja se oscurece.

104 NUEVO	`.cancelar {`	Estilo del enlace **Cancelar**.
105 NUEVO	`margin-top: 10px;`	10 px de espacio arriba.
106 NUEVO	`text-align: center;`	Texto centrado (funciona porque, dentro del formulario en columna, el enlace ocupa todo el ancho).
107 NUEVO	`color: #333;`	Color gris oscuro.
108 NUEVO	`text-decoration: underline;`	Subrayado, para que se vea como enlace.
109 NUEVO	`}`	Cierra la regla.
111 NUEVO	`/* Fila del contacto que se está modificando */`	Comentario.

112–113 NUEVO	`.tabla tr.seleccionado,`
	`.tabla tr.seleccionado:nth-child(even)`
	`{`
		Dos selectores: la fila con clase `seleccionado` y esa misma fila cuando sea par. Se necesita el segundo porque la regla de filas pares (línea 84) es más específica y, sin esto, pintaría de gris la fila seleccionada en lugar de amarillo.

114 NUEVO	`background-color: #fff3cd;`	Fondo amarillo pálido para resaltar el contacto que se está editando.
115 NUEVO	`font-weight: bold;`	Texto en negritas.

Línea	Código	¿Qué hace? (explicación)
116 NUEVO	}	Cierra la regla.

5. Glosario de palabras clave

Término	Significado
Variable	Una cajita con nombre donde se guarda un dato. Ej.: `nombre = "Ana"`.
Función	Una receta con nombre (`def`) que se puede usar muchas veces.
Diccionario	Colección de pares etiqueta: valor, como `DB_CONFIG`.
Tupla	Lista de valores entre paréntesis que no se puede cambiar. Ej.: `(nombre, telefono)`.
Ruta	Una dirección web (`/`, `/editar/3`) asociada a una función de Python.
Decorador	La línea con `@` sobre una función; aquí conecta la función con una dirección web.
GET / POST	Dos formas de pedir algo al servidor. GET = solo consultar (abrir una página); POST = enviar datos (un formulario).
Base de datos	Programa que guarda información de forma ordenada y permanente (aquí, MySQL).
Tabla / fila / columna	Como una hoja de Excel: la tabla es la hoja, la fila un registro (un contacto), la columna un dato (nombre).
SQL	Lenguaje para dar órdenes a una base de datos: `SELECT`, `INSERT`, `UPDATE`, `DELETE`.
Cursor	El "mensajero" que envía órdenes SQL a MySQL y trae las respuestas.
Commit	Confirmar y guardar definitivamente los cambios en la base de datos.
Plantilla (template)	HTML con huecos que Flask rellena con datos de Python.
Jinja	El lenguaje de las plantillas: `{{ }}`, `{% if %}`, `{% for %}`.
Etiqueta HTML	Marca como `<p>` o `<div>` que le dice al navegador qué es cada cosa.
Clase (CSS)	Un nombre que pones a un elemento (`class="tabla"`) para darle estilo.
Selector (CSS)	La parte de una regla que indica a qué elementos se aplica.
None	El valor de Python que significa "nada" o "sin valor".
Inyección SQL	Ataque que consiste en escribir código SQL en un campo para manipular la base de datos. Los `%s` lo evitan.

6. Buenas prácticas y siguientes pasos

Seguridad: cuida tu contraseña
En `app.py` la contraseña de MySQL está escrita dentro del código. Si compartes o subes el archivo (por ejemplo a GitHub), cualquiera podrá verla. Una mejora es guardarla en una **variable de entorno** y leerla con `os.environ.get("DB_PASSWORD")`. Además, si esa contraseña ya se compartió, conviene cambiarla.

Ideas para practicar

- **Eliminar contactos:** crea una ruta `/eliminar/<int:id>` con `DELETE FROM contactos WHERE id = %s` y un botón en la tabla.
- **Buscar:** agrega un campo de búsqueda que use `WHERE nombre LIKE %s`.
- **Mensajes:** con `flash()` de Flask, muestra "Contacto actualizado" después de guardar.
- **Cambiar colores:** en `style.css` modifica los códigos hexadecimales y observa qué cambia (¡la mejor forma de aprender CSS!).
- **Modo depuración:** antes de publicar tu sitio, cambia `debug=True` por `debug=False`.